FRED TALKS

8th November 2026

CONTENTS

Writing as a thinking tool

NOREPLY@BLOGGER.COM (MURAT) · 1670 WORDS

Zen of AI Coding

YOAV AVIRAM · 1912 WORDS

Agentic Monitoring

BEN CONGDON · 1924 WORDS

Writing as a thinking tool

NOREPLY@BLOGGER.COM (MURAT) · 07 NOV 2026 · SOURCE

In middle school I could not get through long passages no matter how hard I tried. I would read an entire page and find that nothing had registered, so I started taking notes out of desperation. This was the late 80s, and Tony Buzan's mind maps had just reached Turkey. This led me to produce pages and pages of notes that looked like spiderwebs. Creating those notes have helped me tremendously. I made sense of what I was reading by figuring out which branch they would hang off in my mind maps. I realized my pages were not for storing the information as much as they were for processing it.

It took me another ten years to understand why this works so well. During a stretch of PhD procrastination, I read Manuel Blum's advice to graduate students:

> You are all computer scientists.
>
> You know what FINITE AUTOMATA can do.
>
> You know what TURING MACHINES can do.
>
> For example, Finite Automata can add but not multiply.
>
> Turing Machines can compute any computable function.
>
> Turing machines are **incredibly more powerful** than Finite Automata.
>
> Yet the only difference between a FA and a TM is that
>
> the TM, unlike the FA, has paper and pencil.
>
> Think about it.
>
> It tells you something about the power of writing.
>
> Without writing, you are reduced to a finite automaton.
>
> With writing you have the extraordinary power of a Turing machine.

This is a crisp technical claim about which problems you can solve with writing. If you take the paper away, you downgrade yourself by an entire complexity class!

I think this explains a lot about human history as well. Language let us externalize some of our thinking, and our species got smarter. Writing let us externalize even more of it, and we got smarter again. I suspect this is also why LLMs are as capable as they are: so much of our intelligence is already deposited in language that a model trained on it inherits a good deal of intelligence for free.

Out-of-core execution

When we write, we are externalizing our thinking onto paper, and that is what raises our capability to think bigger thoughts. I liken this to out-of-core execution, or external memory algorithms.

A good example of this for me is reviewing a paper. When I read a paper, I think I understand it, and then I sit down to write the review and find out I understood maybe 60% of it. While writing the review, I end up contextualizing the paper better, and get to understand it fully. And sometimes I even realize the ideas are bigger than the authors noticed, and they connect to many other things.

Sometimes the unearthing goes further than understanding one paper. In 2018, I wrote a quick post about the anatomical similarities between Paxos and Nakamoto consensus. I had not started with that thesis originally. I started writing, and the connections got clearer as I went, and that post became the seed for one of our research papers. This is not uncommon, this is called freewriting. This is what happens when you give the machine a tape.

Writing is also the reason I remember anything. I refer back to my own blog constantly. Because I strained my brain writing those posts, rereading them re-fires the same neurons, and I reconstruct the understanding quickly. In this sense, the blog became my external memory. The thoughts I never wrote down do not stick well, because they were never fully thought.

> If you think without writing, you only think you're thinking.
>
> -- Leslie Lamport

Writing in the age of LLMs

Is writing for thinking getting obsolete now since you can now argue anything with a model that has read everything?

Well, the model has read everything except what is in your brain! It knows the literature, the standard framing, and the consensus. But it does not know your half-formed intuition about why the standard framing may be wrong for this particular case. AI made a lot of progress, but I don't trust an LLM to take the creative leap and make a new connection. I think that would require another AI breakthrough. So, if you open the chat immediately on anything, you lose the opportunity to write/think for yourself.

This is a real problem, and it seems to be spreading quickly. We have been celebrating Amazon's six-pager as a great example of writing-for-thinking. Bezos had said: "There is no way to write a six-page, narratively structured memo and not have clear thinking." Unfortunately, with LLMs, that sentence has stopped being true. You can now produce six pages with no thinking at all, and the six pages still look like evidence that the thinking happened.

A model optimizing next-token probability cannot do your thinking. If you trust it to hold the pen, no thinking would get done. As everyone turns their thinking down and becomes a meat-proxy for generated text, if you still keep writing yourself, you would still be thinking for yourself. So writing is now a comparative advantage, and a widening one!

Anyways, enough about why you should write. Here is my advice on how you should write.

Rule 1: Mess up

The first rule of writing club is to produce in bulk with no restraints at all. Don't criticize, don't fix the typos, don't write in full sentences if you don't feel like it. The goal in this phase is to just accumulate text and get to a zeroth draft you can work with.

McPhee argues best for why getting anything down matters so much. Even if you had a bad writing session, it gave your mind something to work with. After you get something down, you can spot problems with it and think of better ways of phrasing things. But none of that happens without the drafted version to react against.

This is also how research papers get written. Open a notes.org file and treat it as a lab notebook: write while you do the work, explore every tangent, produce without any pressure to be presentable. In the Köroğlu post I compared this to the foal fattened in a pitch-black stable before any training begins. The key part of that frame is the stable. During this phase you guard the idea from critics, including and especially yourself. Ideas start out fragile and easily squished. So suppress judgment for a while and see what the thing becomes.

Rule 2: Tidy up

Now you must become the critic, the editor. Forget that you wrote this. Imagine some other guy wrote it, and do a harsh review. Cut ruthlessly. Take delight in deleting text the way you would in deleting code. If you cannot bear to delete a good sentence, move it to a junk buffer at the bottom of the file so you can tell yourself you might bring it back. Spoiler: you won't.

Don't be discouraged by how bad the first draft was. McPhee says the way to do a piece of writing is three or four times over, never once. This is good news. Thinking in multiple drafts takes the pressure off: you don't have to be spontaneous, brilliant on the first pass, or finished today.

I think the reason writing works so well for thinking is that it slows you down while keeping you on the problem. In that extra time, the insights and connections get room to surface. Chunking/abstracting happens in the background. Editing is then the magic trick where you pretend you had an organized thought all along. It is the packaging of the product.

Rule 3: Keep it simple, plain, and succinct

I am an Elements of Style guy. Strunk and White got it right, and their writing still reads fresh decades later. I adore Vonnegut too. His writing is simple as fuck and completely eloquent. It took a lot of balls to write that plainly. Complexity in prose is usually a place to hide sloppy thinking, or a way to weasel out of taking a stance. If you can't say it plainly, you probably haven't thought it through yet. And the way you think it through is by writing/editing.

Rule 4: Write without expectations

Write for yourself. The audience I have in mind is me. If others read it, that is a bonus, but the motivation has to be intrinsic, because the external rewards are far too delayed and too random to steer on. Writing like no one is watching also frees you up, and you can decide afterwards how much to share. Get your dopamine hit from the writing itself, not from people reading or sharing what you wrote, as that is very unreliable.

I have been writing and sharing on this blog since 2011, and around 2016 I started running into people at conferences who read it. I assumed it was a fluke, but it kept happening. Writing with a human voice turns out to get a lot of attention, and I am still surprised by how much. Paradoxically, you seem to get an audience only if you weren't doing it for the audience.

Rule 5: Put it on the schedule

You should stop trying to get inspiration to write and start scheduling it. Waiting for inspiration doesn't work, instead, putting your butt on seat and writing breeds good ideas for writing.

I find that keeping a posting schedule for the blog keeps me in "a writing frame of mind", which is really "a noticing frame of mind". Many weeks I tell myself a topic isn't interesting, but I start writing about it anyway, and then, what do you know, I am startled by the fount of interesting things that surfaces.

I also feel relieved after writing something as the thought is out of my system, I have closure on it, and I have room to think new thoughts. If a week goes by without a post, I feel clogged. So put writing on the calendar and keep it there until it no longer needs to be on the calendar.

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · SOURCE

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to Zen of Python by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead
2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack

8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world
11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus’s law states that “given enough eyeballs, all bugs are shallow.” Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How “dense” are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We’ve built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is tricker then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering
We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy
I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of WhiteBoar, where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.

Agentic Monitoring

BEN CONGDON · 23 OCT 2026 · SOURCE

A recent Anthropic blog post contained a short paragraph, not at all the point of the post, which has been buzzing around my brain since I read it:

> A new level of operational excellence is possible. Our team uses Claude Tag heavily, which has helped us monitor each component of the system. For example, we created Slack channels where we tasked Claude with being obsessively focused on the health of a single component. We call these Claudes "superfans." We have a scion-daemon superfan, a scion-cache superfan, an Antique merge queue superfan, and so on. They

> have access to our telemetry and logs, and act like an experienced engineer who has nothing better to do than to refresh charts and look for anything strange. We also have traditional alerts for all of our systems, but our superfans can detect subtle and novel issues that make these alerts seem one-dimensional by comparison.

This is *so obviously* the path forward for software systems operations. I've been thinking about monitoring with agents for the past few weeks, and this paragraph crystallized some of those thoughts.

Modern software systems, like the LLMs that write them, are becoming more "grown" than "designed". Yes, human engineers often still give the high-level design and shape to the system, and yes, software systems (for now) are much more interpretable than LLMs – since you can just sit down and read the code, if you are so inclined. However, with the volume of code being written and shipped increasing exponentially, the amount of actual eyes-on-code knowledge that any human operator of a system has is decreasing.

Systems monitoring was never *easy*. There is a whole industry of observability SaaS companies – Sentry, Datadog, Splunk, Honeycomb, to name a few. Observability is expensive.[1] Doing it well requires a specialized human skill set and relies on building experience operating real systems at scale. One can really only develop these skills by working in industry. This is a domain that takes years to train junior engineers on.

As with seemingly everything in the software industry in 2026, system monitoring is poised for a step change due to agents. I… am surprised I have not seen more people write about this, especially in the current milieu of "agent frenzy". It's right there in front of us all, and it *works*. There have been a bunch of startups trying to do the "AI SRE", but very little writing from practitioners.

But you don't really need a full "AI SRE" to improve operational excellence, nor do you need a built-out SaaS product to do so. As of the Opus 5.5 / Astra class of models, with a bit of care, it's entirely possible to make a good agent-driven monitoring system that "just works".

Case study: change-point monitoring

I'll ground this discussion in a problem that is near to my heart: production monitoring of change points, like binary releases and config pushes. The idea is that you want to do specialized, intensive monitoring around critical points of change to a system; these change points are more likely to trigger a regression, and thus if you monitor these events closely, you can limit its impact. To quote the clichéd line from the Google SRE book, "~70% of outages are due to changes in a live system".

This is a wicked problem. I've been working on it in some form since 2023, and it's been hard to get traction on. From talking with folks across the industry, I've learned there are ways to do change-point monitoring "well", but these methods largely still rely on humans to do the right thing (e.g. tagging metrics/alerts diligently, defining precise signals, etc.) and require significant organizational investment.

More specifically, achieving "takeoff" for change-point monitoring as a piece of platform engineering is hard because:

- Your first instinct may be to target highly motivated users who already have a good ops posture. However, these teams' rollouts are already mostly safe, and you don't actually raise the ops waterline much. The thing you're selling here is *toil reduction*, which is "nice to have" but is not itself a first-order safety improvement. Precision is easier to achieve for highly motivated teams, but this means the change-point system is "competing" on recall and latency with traditional alerts.

- You can alternatively target the broad population of services. In other words, you go super wide and opt in "all services", on a chosen subset of universal health metrics. In practice, most of these general health metrics are irreducibly noisy. This noise results in poor precision and alert fatigue.

- Prior to LLMs, there was a clear precision/latency tradeoff as well. Along this spectrum are: structured logs (precise, slow), metrics (coarse, fast), and alerts (even coarser, recall is pretty bad and precision is pretty bad, but "at least it's something").

Shockingly, agents largely "just" solve this problem. Agents are happy to run copious queries, can write per-change queries on the fly, and will scan logs ad nauseam. As a result, you can get both good precision and recall for cheap. In the basic version of this setup, you just need the ability to connect your codebase (e.g. to reason about config consumption) and your observability stack (e.g. to view and monitor the live production state) to the agent.

I've spiked a prototype of exactly this: an agent that proposes a monitoring plan based on the service code, a scaffold that executes that plan, and an agent that investigates triggered monitors, with an escalation path to humans. Even in this early prototype, I've been able to detect regressions in systems I have *zero* understanding of with precision and latency that would have been impossible a year ago.

In my iteration on this, I've found that a pure agent loop is not the optimal shape for agentic monitors. Rather, a harness that allows the agent to write a spec of how to monitor, along with a deterministic system for running those specs, produces significantly cleaner signals with

dramatically lower token usage. To the extent there is “secret sauce” in this pattern, it’s in the noise-reduction heuristics, the prompts that get the agent to write good specs, and a labeled eval set large enough to validate further tweaks.[2]

Agents are structurally well suited to monitoring

Two mindset shifts need to happen before this pattern can catch on:

1. Agents actually can do a much better job at monitoring than humans.
2. Monitoring agents need scaffolding, the same way the rest of the software development lifecycle had to adapt for coding agents. There hasn’t been a “Claude Code moment” for monitoring yet.

Once you make these shifts, monitoring agents have several “unfair advantages” over human operators:

- Ability to track everything, 24/7. If tokens are cheap, then any regression in a metric you care about and any suspicious log line can be investigated – not just the events that would previously have risen above the “human attention cutline”. Agents don’t need to “follow the sun”.
- Ability to fan out and investigate several paths toward resolving a production problem. Agents can parallelize investigations when multiple things go wrong at once, and coordinate quickly. This is already my go-to for incident triage: I just launch a bunch of agents to investigate several potential threads or theorized incident causes. Right now, this is human-driven, but there’s really no reason it couldn’t be agent-initiated in the future. You can elastically scale up a fleet of incident responders.
  - We’re already seeing that this is becoming critical for security incidents. Teams/fleets/swarms of agents are being used for offensive cyberattacks, and defenders are beginning to build up similar defensive capabilities (e.g. OpenAI’s Daybreak Blue, Anthropic’s Project Glasswing, Google’s Fairwind).
- Ability to respond faster to a problem than can a human. Agents can read code faster than humans. Agents can also query and digest metrics and log signals significantly faster. This speed doesn’t always translate into faster incident detection time; a human with a deep

understanding of a system may still be faster at finding the right signal and intuiting the root causes. But to the extent that tacit knowledge can be recorded and made available to an agent, the agent will eventually beat the human on speed.
    - By extension, for systems where the human operator isn't deeply experienced, the "time to understanding" is going to be shorter for an agent.

"A new level of operational excellence is possible."

Be more diligent: You wouldn't pay a full-time software engineer to look through all the Git commits for a service, pick out things to monitor, and make special short-term fast-trigger alerts just on those changes. However, for a couple cents per million tokens, it's possible now, and will become an industry standard.

Continuously improve performance: Agents can monitor performance, look at heap dumps and stack traces and CPU profiles, and propose optimizations from the patterns they find. Put the proposed improvements behind A/B tests if necessary, and monitor as you would any other change to the service.

In one instance, I observed that a few sessions of Astra/Opus investigation discovered 50% heap savings and 30% latency reduction in a common hot-path library that humans had already spent nontrivial effort to optimize. Once you give a sufficiently powerful LLM a VM and benchmarks to run, the "hill-climbing" ability of current-generation LLMs is remarkable.

Take action: Thus far, I've mostly been discussing "read-only" monitoring systems. Agents, of course, needn't be "read-only". With the right harness and guardrails in place, there's definitely a future in which agents will be able to take front-line actions to remediate incidents. This could span anything from well-scoped operational tasks like resizing instances or rotating compute pools, to deterministic actions like rolling back a config change or binary, to open-ended tasks like root-causing bugs, proposing fixes, and redeploying on-the-fly. Obviously there is risk the more control you hand-over to the agent, but considering how much agent capabilities expanded over the course of the past 12 months, the amount of write-path work that can be responsibly handed over to agents is bound to increase. The promise of truly self-healing systems is exciting.

Cost is no longer a limitation. Luna/Haiku-class models[3] are now competent graph-starers. A moderately large tech company could likely monitor its entire fleet for on the order of $100/day. Startup-scale fleets would be a small fraction of that. Compared to human labor, or even the cost of the infra that runs observability, this is trivial.

Create a virtuous cycle: If done well, agentic monitoring creates a positive feedback loop: coding agents allow you to ship more, monitoring agents allow you to support more systems, which in turn frees up human time to direct more things to ship. Humans increasingly spend time "building the factory" or "tending the garden".

Agentic monitoring is worth investing in. It will soon be reckless not to have agents monitoring your systems. Until the rest of the industry catches up, getting started now affords an unfair advantage.

1. A set of optimizations my team recently completed saved us several million dollars in annualized observability cost, and… that was a mere fraction of what *just my team* spends on observability. ↩

2. Much, much more could be written about the design of agentic monitoring harnesses. This is one of the clearest "we're so early" feelings I've had in a while. General harnesses, like Anthropic's "Claude Tag", evidently already do a good job at agentic monitoring, per the original post. However, based on what I've seen, even a little monitoring-specific harness engineering picks up a lot of low-hanging fruit. ↩

3. Here I mean GPT-6-Luna and Claude Haiku 5.5. ↩